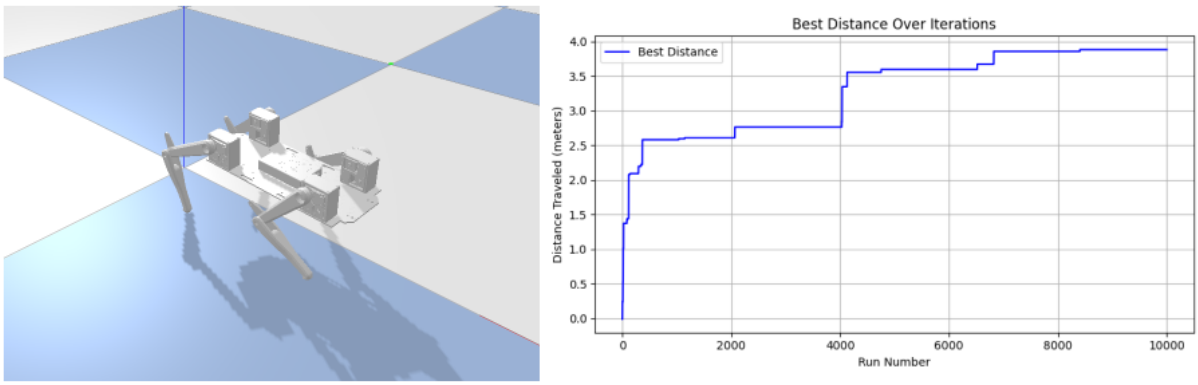
Trot or Rot: Applying RL to Quadruped Locomotion

Maksym Bondarenko
mb5018@columbia.edu



Abstract

This report analyzes the application of reinforcement learning (RL) to quadruped locomotion. Specifically we focus on looking at minitaur robot in simulation and trying to teach it to walk/run. The project focuses on analyzing different RL algorithms performance in simulation, and what considerations go into applying RL effectively to quadruped robots. Our code is available at https://github.com/Lenguist/trot-or-rot

1. Introduction

Quadruped locomotion allows robots to navigate environments inaccessible to wheeled vehicles, from disaster zones to rough outdoor terrain. After decades of research, quadruped robots now walk reliably across various surfaces, with companies like Boston Dynamics (Spot) [2], Unitree Robotics (Go1) [13], and Ghost Robotics [12] bringing these capabilities to market. Reinforcement learning (RL) drove much of this progress, replacing brittle hand-crafted controllers with adaptable learned policies [7, 8, 16].

This project investigates why RL transformed quadruped locomotion and what factors affect its performance. We implement and compare four algorithms of increasing complexity—Hill Climbing, Vanilla Policy Gradient (VPG), Proximal Policy Optimization (PPO) [15], and Soft Actor-Critic (SAC) [3]—using the Minitaur robot in PyBullet simulation. Through systematic experiments, we analyze how these algorithms perform across metrics including speed, stability, and sample efficiency.

Rather than introducing novel techniques, we focus on understanding fundamentals: which approaches work best in different scenarios, how hyperparameter choices affect learning outcomes, and what practical considerations matter when applying RL to legged locomotion. These insights help bridge the gap between theoretical understanding and practical implementation of learned controllers for robotic systems.

Our primary contributions are:

1. A comparative analysis of RL algorithms for quadruped locomotion, highlighting their relative strengths, weaknesses, and computational requirements
2. An investigation of hyperparameter sensitivity across al-

gorithms, revealing which factors most strongly influence locomotion performance
3. Insights into the practical considerations for applying RL to robotic locomotion, particularly regarding reward function design and training protocols

2. Related Work

Quadruped locomotion research has evolved from handcrafted controllers to learning-based approaches. Traditional methods used predefined gaits but struggled with unexpected conditions. Early optimization techniques like hill climbing [5, 6] showed promise but lacked adaptability to varied terrain.

Physics simulations transformed this landscape by enabling safe, efficient training. PyBullet [1] and similar environments [4, 9] allowed training through millions of interactions before hardware deployment. Tan et al. [16] demonstrated this by successfully transferring policies from simulation to the physical Minitaur robot, though certain gaits required careful initialization.

The reality gap between simulation and physical robots presented significant challenges, addressed through domain randomization [10] and improved system identification [16]. As basic locomotion improved, research shifted to challenging environments. Lee et al. [8] achieved a breakthrough with their ANYmal robot traversing mud, snow, and rubble—environments never encountered during training.

Recent advances focus on data efficiency and adaptability. Yang et al. [17] reduced data requirements by an order of magnitude using model-based RL. Kumar et al. [7] developed a two-part architecture allowing robots to quickly adjust to changing conditions. Peng et al. [11] demonstrated imitation learning from animal movements, bypassing complex reward design.

Algorithm development has been equally important. Trust Region Policy Optimization [14] and its successor PPO [15] provided stable policy updates, while Soft Actor-Critic [3] improved sample efficiency through maximum entropy RL.

RL has become dominant through converging developments: improved simulations, curriculum learning enhancing generalization, and specialized algorithms handling high-dimensional spaces. Our work systematically compares algorithms across the complexity spectrum on identical tasks, revealing their practical strengths and limitations.

3. Methodology

3.1. Environments

3.1.1. Custom Quadruped Robot

For my initial experiments, I worked with a custom quadruped robot developed in a previous class. This robot features four legs, each with multiple degrees of freedom. The observation space consists of IMU measurements (providing orientation and acceleration data) and joint angle readings from each leg. The action space allows for direct control of joint angles.

The environment was implemented in PyBullet, providing physics simulation for the robot's interaction with the terrain. Despite having a functioning environment, applying RL algorithms to this custom setup proved challenging due to the lack of standardized wrappers and established benchmarks.

3.1.2. Minitaur Environment

The Minitaur environment simulates a quadruped robot with eight direct-drive motors using the PyBullet physics engine. The environment follows the OpenAI Gym interface and simulates the Ghost Robotics Minitaur, a small quadruped robot with articulated legs.

The observation space is 28-dimensional, as shown in the MinitaurGymEnv implementation:
- Joint angles for all 8 motors (8 values)
- Joint angular velocities for all 8 motors (8 values)
- Motor torques for all 8 motors (8 values)
- Base orientation as a quaternion (4 values)

The action space is 8-dimensional and continuous, with each dimension bounded in the range $[-1, 1]$, representing target joint angles for the eight motors. The environment can optionally use a leg model that reparameterizes these actions based on the robot's kinematics.

The reward function has four components, weighted by configurable parameters:

$$R = w_d \cdot r_{\text{forward}} + w_e \cdot r_{\text{energy}} + w_{\text{drift}} \cdot r_{\text{drift}} + w_s \cdot r_{\text{shake}} \quad (1)$$

Where:
- r_{forward}: Reward for forward displacement (current x position - previous x position)
- r_{energy}: Penalty for energy consumption, calculated as the dot product of motor torques and velocities
- r_{drift}: Penalty for sideways displacement (absolute change in y position)
- r_{shake}: Penalty for angular motion that deviates from upright orientation

For our experiments, we use weights $w_d = 1.0$, $w_e = 0.005$ (Hill Climbing) or $w_e = 0.008$ (PPO), $w_{\text{drift}} = 0.0$, and $w_s = 0.0$, prioritizing forward locomotion while moderately penalizing energy consumption.

An episode terminates when the robot falls over (determined by orientation or height) or exceeds a distance limit. For our experiments, we use episodes of 1000 timesteps with a control step of 0.01 seconds.

3.1.3. Half-Cheetah Environment

After encountering challenges with the Minitaur environment, I transitioned to the simpler and more widely stud-

ied Half-Cheetah environment. Half-Cheetah is a 2D robot consisting of 9 links and 8 joints, with 6 actuated joints. It has become a standard benchmark in RL research due to its moderate complexity and clear evaluation metrics.

The observation space is 17-dimensional, consisting of:

- Position and angle measurements (8 values)
- Velocity measurements (9 values)

The action space is 6-dimensional and continuous, representing the torques applied to the six actuated joints. Each action value is bounded in the range $[-1, 1]$.

The reward function in Half-Cheetah has two components:

$$R = r_{\text{forward}} - 0.1 \cdot r_{\text{ctrl}} \tag{2}$$

Where:

- r_{forward}: Reward for forward movement, measured as the change in x-coordinate
- r_{ctrl}: Control cost, calculated as the squared sum of actions

This reward structure encourages the agent to move forward while using minimal energy.

3.2. Algorithms

3.2.1. Hill Climbing (Baseline)

Hill Climbing serves as our simplest baseline. Unlike the other algorithms which learn to map observations to actions, Hill Climbing optimizes a compact parameterization of the gait directly.

We use a sinusoidal parameterization where each joint angle is determined by:

$$\theta_i(t) = a_i \cdot \sin(2\pi f t + \phi_i) \tag{3}$$

For 8 joints, this results in 17 parameters: amplitude a_i for each joint (8 parameters), phase offset ϕ_i for each joint (8 parameters), and a single shared frequency f. The algorithm works as follows:

Algorithm 1 Hill Climbing for Minitaur Locomotion

Initialize parameters θ randomly (scaled by 0.5)
Evaluate fitness of θ by running episode
while not reached 5000 iterations **do**
  $\theta' \leftarrow \theta + \mathcal{N}(0, \sigma)$ {Add Gaussian noise with $\sigma = 0.1$}
  Evaluate fitness of θ' by running episode
  if fitness(θ') ¿ fitness(θ) **then**
    $\theta \leftarrow \theta'$ {Accept if improvement}
  end if
end while

The fitness function is the cumulative reward over a 1000-step episode. This simple approach allows us to find effective gaits with minimal computational requirements, serving as a baseline to compare against more sophisticated RL methods.

3.2.2. Proximal Policy Optimization (PPO)

Proximal Policy Optimization builds upon policy gradient methods by introducing mechanisms to ensure stable policy updates. It uses a clipped surrogate objective to limit the policy change per iteration, preventing large destructive updates.

We implement PPO using the Stable Baselines3 library with the following network architecture:

- **Policy Network**: A multilayer perceptron with two hidden layers (185 and 95 units) using tanh activations.
- **Value Network**: A separate network with two hidden layers (95 and 85 units) predicting the state value function for advantage estimation.

The algorithm is trained as follows:

Algorithm 2 Proximal Policy Optimization (PPO)

For each iteration:
  Collect trajectories by executing the current policy π_θ
  Compute advantage estimates $\hat{A}_t$
  For $k = 1$ to K epochs:
    Compute the ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
    Compute clipped objective: $L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$
    Update θ by maximizing $L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 S[\pi_\theta]$

For our implementation, we use hyperparameters: batch size of 1000, 10 optimization epochs per iteration, clip range of 0.2, learning rate of 3×10^{-4}, discount factor $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, and value function coefficient $c_1 = 0.5$. We train across 25 parallel environments for a total of 7 million timesteps.

While I implemented the theoretical framework for all these algorithms, I ultimately focused my practical experiments on Hill Climbing (as a baseline) and PPO due to time constraints and the challenges encountered with environment stability.

4. Experiments and Results

4.1. Challenges with Custom Quadruped Environment

My initial experiments with the custom quadruped robot revealed significant challenges in applying RL to custom robotics environments. I found that while the robot could be simulated in PyBullet, the lack of standardized gym wrappers made it difficult to apply modern RL algorithms effectively. The PPO implementation struggled to learn useful behaviors, often resulting in erratic movements or the robot falling over immediately.

This experience highlighted the importance of standardized interfaces and well-tested environments when applying RL to robotics problems. After several unsuccessful attempts to create a robust wrapper, I decided to move to the more established Minitaur environment.

4.2. Minitaur Environment Results

For the Minitaur environment, I was able to successfully implement a Hill Climbing baseline that learned effective walking gaits. The Hill Climbing algorithm, despite its simplicity, was able to find parameters that produced a stable forward walking motion after a few hundred iterations.

Figure 1 shows the learning progress of the Hill Climbing algorithm on the Minitaur environment. The algorithm quickly improves from negative rewards to positive values within the first few hundred iterations, though it plateaus relatively quickly afterward.

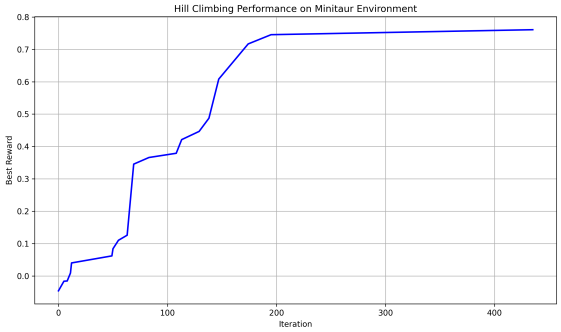


Figure 1. Performance of Hill Climbing algorithm on Minitaur environment. The algorithm shows rapid improvement in the first few hundred iterations before plateauing around reward value 4.0.

However, my attempts to apply PPO to the same environment were less successful. Despite extensive tuning of hyperparameters and multiple training runs of up to 4 million steps, the algorithm consistently failed to achieve positive rewards. The training would show initial improvement from large negative values (around -10,000) to smaller negative values (around -0.3), but would never cross into positive territory.

This stark contrast between the performance of Hill Climbing and PPO was surprising and led me to investigate several possible causes:

- **Observation/Reward Normalization**: Research into similar issues revealed that a common problem in applying RL to locomotion tasks is the lack of proper normalization of observations and rewards. The high dynamic range of sensor values and rewards can significantly impede learning.
- **Dimensional Complexity**: The Minitaur environment has a high-dimensional observation and action space, which can make exploration difficult for policy gradient methods. The original Sim2Real paper for Minitaur mentions reducing the observation space to learn effective gaits.
- **Implementation Challenges**: Despite using established libraries like Stable Baselines3, there may be subtle implementation details or compatibility issues between the library and the PyBullet environment.

4.3. Half-Cheetah Environment Results

To isolate the source of the problems encountered with Minitaur, I conducted experiments with the simpler Half-Cheetah environment using PPO implementations from Stable Baselines3. A critical insight was the importance of observation and reward normalization for these environments.

Figure 2 shows the learning curves for PPO on the Half-Cheetah environment, comparing performance with and without normalization.

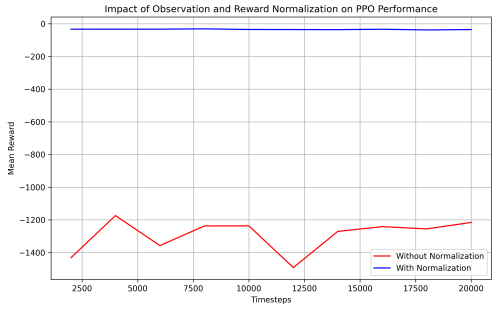


Figure 2. Impact of observation and reward normalization on PPO performance in Half-Cheetah environment. With normalization (blue line), the algorithm shows consistent learning progress, while without normalization (red line), it struggles to make meaningful improvements.

The results clearly demonstrate that normalization is crucial for effective learning in these environments. Without normalization, the algorithm struggles to make meaningful progress, whereas with normalization, it shows steady improvement in performance.

This finding suggests that the challenges encountered with the Minitaur environment may indeed be related to normalization issues rather than fundamental limitations of the algorithms themselves.

5. Discussion and Limitations

This project revealed several important insights about applying RL to robotics problems, particularly for locomotion tasks. While I encountered significant challenges that limited the scope of my final results, these challenges themselves provided valuable learning opportunities.

5.1. The Critical Role of Environment Design

Perhaps the most significant insight from this project is the importance of properly designed simulation environments for successful RL application. Key factors include:

- **Observation and Reward Normalization**: My experiments with the Half-Cheetah environment clearly demonstrated the critical impact of normalization on learning performance. This appears to be a common issue in locomotion tasks where sensor values and rewards can have high dynamic ranges.
- **Standardized Interfaces**: The contrast between working with custom environments versus established benchmarks highlighted the value of standardized interfaces like the OpenAI Gym API.
- **Observation Space Design**: The dimensionality and information content of the observation space significantly impacts learning efficiency. This was apparent in the challenges with the high-dimensional Minitaur environment compared to the more manageable Half-Cheetah environment.

5.2. Algorithm Selection and Implementation

While modern algorithms like PPO and SAC have shown impressive results in the literature, my experience suggests that:

- **Simpler Methods Can Be Effective**: The Hill Climbing algorithm was surprisingly effective for the Minitaur environment, despite being much simpler than policy gradient methods. This suggests that for some well-structured problems with compact parameterizations, simpler optimization approaches may be sufficient.
- **Implementation Details Matter**: Small implementation details like normalization, network architecture, and hyperparameter settings can have outsized impacts on performance. The theoretical guarantees of algorithms like PPO are contingent on proper implementation.
- **Domain-Specific Adaptations**: Successful applications of RL to robotics often involve domain-specific adaptations that aren't always apparent from the core algorithm descriptions.

5.3. Limitations

This project had several limitations that should be acknowledged:

- **Time Constraints**: The time available for experimentation limited the scope of hyperparameter tuning and the number of environments and algorithms that could be thoroughly tested.
- **Computational Resources**: Training RL agents for locomotion tasks requires significant computational resources, particularly for high-dimensional environments like Minitaur.
- **Implementation Challenges**: Debugging subtle issues in RL implementations can be extremely time-consuming, especially when the problems manifest as poor learning performance rather than explicit errors.
- **Limited Baselines**: With more time, establishing stronger baselines and conducting more systematic comparisons would have provided clearer insights.

6. Conclusions

This project set out to explore the application of reinforcement learning algorithms to robotic locomotion tasks. While the practical results fell short of my initial ambitious goals, the process yielded valuable insights into the challenges and requirements for successful application of RL in robotics.

The key findings from this work include:

- **Environment Engineering is Critical**: Many of the advancements in applying RL to robotics come not just from algorithm improvements, but from better simulation environments, observation space design, and reward shaping. The importance of normalization in particular was clearly demonstrated in my experiments.
- **Algorithm Selection Should Match Problem Complexity**: While sophisticated algorithms like PPO and SAC offer powerful capabilities, simpler methods like Hill Climbing can be surprisingly effective for well-structured problems with compact parameterizations.
- **Practical Implementation Challenges**: The gap between theoretical RL algorithms and their successful practical implementation remains significant, with many subtle details that can dramatically impact performance.

Going forward, I plan to continue this work beyond the course deadline, focusing on resolving the normalization issues in the Minitaur environment and conducting more systematic comparisons between algorithms. I believe that the insights gained from addressing these challenges will be valuable for future work in applying RL to real-world robotics problems.

Despite the limitations and challenges encountered, I found this project to be an extremely rewarding learning experience. It provided practical insights into the realities of applying RL to complex control problems that could not have been gained through theoretical study alone. I hope that the honest assessment of both successes and failures in this report contributes to a better understanding of the practical considerations in applying RL to robotics.

References

[1] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. http://pybullet.org, 2021. 2

[2] Boston Dynamics. Spot robot. https://www.bostondynamics.com/spot, 2023. Accessed: 2023-05-01. 1

[3] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. pages 1861–1870, 2018. 1, 2

[4] Jemin Hwangbo, Joonho Lee, and Marco Hutter. Per-contact iteration method for solving contact dynamics. *IEEE Robotics and Automation Letters*, 3(2):895–902, 2018. 2

[5] Antonio R Jimenez. *Policy search approaches to reinforcement learning for quadruped locomotion*. Master's thesis, Massachusetts Institute of Technology, 2004. 2

[6] Jihoon Kim, Dang Xuan Ba, Hoyeon Yeom, and Joonbum Bae. Gait optimization of a quadruped robot using evolutionary computation. *Journal of Bionic Engineering*, 18: 306–318, 2021. 2

[7] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. Rma: Rapid motor adaptation for legged robots, 2021. 1, 2

[8] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47):eabc5986, 2020. 1, 2

[9] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance gpu-based physics simulation for robot learning. In *Neural Information Processing Systems (NeurIPS), Track on Datasets and Benchmarks*, 2021. 2

[10] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018. 2

[11] Xue Bin Peng, Erwin Coumans, Tingnan Zhang, Tsang-Wei Lee, Jie Tan, and Sergey Levine. Learning agile robotic locomotion skills by imitating animals. In *Robotics: Science and Systems*, 2020. 2

[12] Ghost Robotics. Vision series quadruped robots. https://www.ghostrobotics.io/, 2023. Accessed: 2023-05-01. 1

[13] Unitree Robotics. Go1 quadruped robot. https://www.unitree.com/go1/, 2023. Accessed: 2023-05-01. 1

[14] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. pages 1889–1897, 2015. 2

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 1, 2

[16] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems*, 2018. 1, 2

[17] Yuxiang Yang, Ken Caluwaerts, Atil Iscen, Tingnan Zhang, Jie Tan, and Vikas Sindhwani. Data efficient reinforcement learning for legged robots. In *Conference on Robot Learning*, 2020. 2